

Lenguaje Python

Características generales

Es un lenguaje orientado a objetos interpretado, adecuado para la creación de scripts. Python es interactivo, se pueden lanzar instrucciones directamente a través del intérprete, aunque también se pueden crear archivos .py y ejecutarlos.

Python se puede ejecutar en una amplia variedad de plataformas, puesto que existen intérpretes para windows, mac, linux, etc.

Además de programación estructurada y orientación a objetos, python también soporta programación funcional.

Python utiliza, al igual que Java, un recolector de basura que se encarga de la gestión de la memoria

Para obtener python podemos descargarlo desde esta dirección:

<https://www.python.org/downloads/>

En ella encontramos la distribución básica. También podemos instalar un entorno más sofisticado como anaconda:

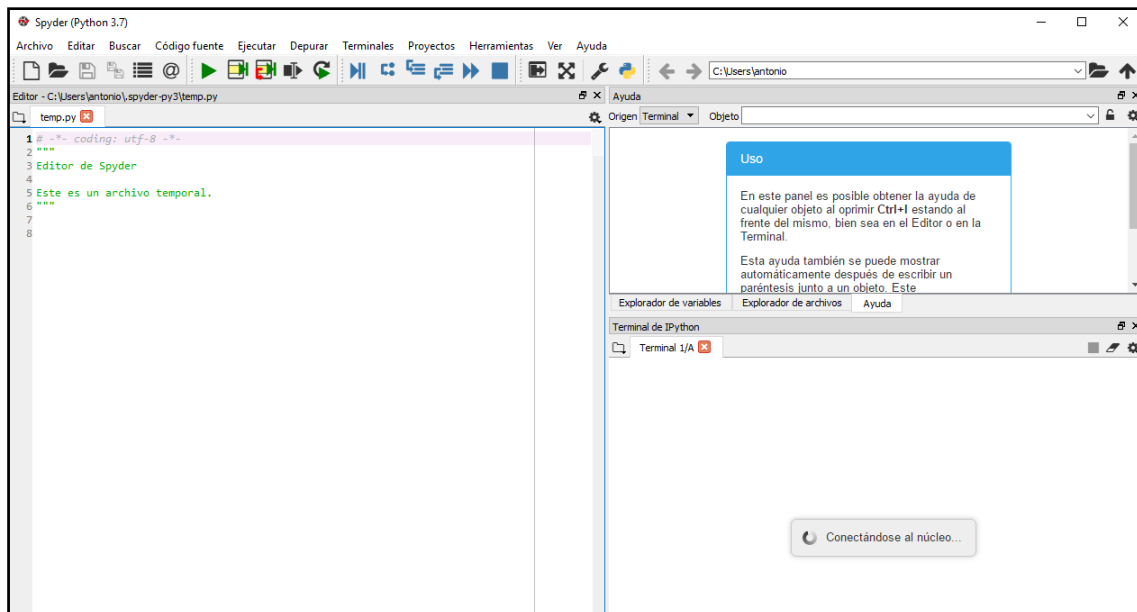
<https://www.anaconda.com/download/>

Anaconda incluye Spyder, que es el entorno de desarrollo para Python.

Para arrancar Python en el modo interactivo, teclearemos desde línea de comandos:

```
>python
```

Si queremos utilizar el IDE Spyder, lo lanzaremos desde el Anaconda Navigator al que accedemos desde el menú de inicio de Windows. Su aspecto será:



Una vez en el modo interactivo, que aparece representado por:

```
>>>
```

podemos empezar a lanzar instrucciones en este lenguaje y ver los resultados, como por ejemplo:

```
>>>print("hola Python")
```

que mostrará

```
>>>hola python
```

Si lo que queremos es ejecutar un script, salimos del modo interactivo con

```
>>>exit()
```

Y ya desde el prompt lanzamos el script:

```
>python c:\test.py
```

Sintaxis básica

Comentarios

Los comentarios en python pueden ser de una línea:

```
#Esto es un comentario
```

o de varias líneas, para lo cual usamos la triple comilla (doble o simple):

```
'''
```

Este comentario ocupa

varias líneas

```
'''
```

Este último caso no es un comentario real, simplemente es una forma de crear una cadena en varias líneas que no se asigna a ninguna variable

case sensitive

Python es case sensitive, no es lo mismo true que True

Identificadores

Los identificadores para variables en python pueden contener cualquier combinación de caracteres alfabéticos, números y subrayado, aunque no pueden comenzar por número:

```
asd=10 #correcto
```

```
_f1=20 #correcto
```

```
_3=3 #correcto
```

```
1ft=4 #incorrecto
```

Las siguientes palabras son reservadas y no pueden utilizarse como nombres de variables:

and	exec	not
assert	finally	or
break	for	pass
class	from	print
continue	global	raise
def	if	return
del	import	try
elif	in	while
else	is	with
except	lambda	yield

Variables y tipos de datos

Variables

En python las variables no se declaran, cuando se necesiten se les asigna directamente el valor (con el operador de igualdad =) y será en ese momento cuando se reserve el espacio de memoria correspondiente, en función del tipo asignado.

Sin embargo, a una variable se le puede asignar un valor de tipo diferente cada vez. En cada asignación se reservaría un nuevo espacio para el almacenamiento del dato:

```
can=100 #se le asigna un entero
```

```
can=5.7 #ahora se le asigna un decimal
```

```
can="hello" #ahora se le asigna un texto
```

Tipos de datos

Python soporta varios tipos de datos, un primer grupo de tipos básicos sería: **entero, coma flotante, boolean y cadena**:

```
num=100 #entero
```

```
dec=5.67 #coma flotante
```

```
bol=True #boolean
```

```
cad="hello" #cadena
```

Se puede realizar una asignación de un mismo valor a varias variables:

```
a=b=c=5
```

Y también varias asignaciones de distintos tipos:

```
a,b,c=2, 4.5, "bye"
```

Se pueden escribir varias instrucciones en una misma línea, separándolas por ;

```
a=10;k="hello";print("bye")
```

Existe otro conjunto de tipos de datos más complejos, como las listas, tuplas y diccionarios, que se estudiarán más adelante.

Operadores

Python soporta los siguientes tipos de operadores:

Aritméticos

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	$a + b = 30$
- Subtraction	Subtracts right hand operand from left hand operand.	$a - b = -10$
* Multiplication	Multiplies values on either side of the operator	$a * b = 200$
/ Division	Divides left hand operand by right hand operand	$b / a = 2$
% Modulus	Divides left hand operand by right hand operand and returns remainder	$b \% a = 0$
** Exponent	Performs exponential (power) calculation on operators	$a ** b = 10 \text{ to the power } 20$
//	Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed. But if one of the operands is negative, the result is floored, i.e., rounded away from zero (towards negative infinity) –	$9 // 2 = 4$ and $9.0 // 2.0 = 4.0$, $-11 // 3 = -4$, $-11.0 // 3 = -4.0$

Condicionales

Operator	Description	Example
==	If the values of two operands are equal, then the condition becomes true.	(a == b) is not true.
!=	If values of two operands are not equal, then condition becomes true.	(a != b) is true.
<>	If values of two operands are not equal, then condition becomes true.	(a <> b) is true. This is similar to != operator.
>	If the value of left operand is greater than the value of right operand, then condition becomes true.	(a > b) is not true.
<	If the value of left operand is less than the value of right operand, then condition becomes true.	(a < b) is true.
>=	If the value of left operand is greater than or equal to the value of right operand, then condition becomes true.	(a >= b) is not true.
<=	If the value of left operand is less than or equal to the value of right operand, then condition becomes true.	(a <= b) is true.

Asignación

Operator	Description	Example
=	Assigns values from right side operands to left side operand	$c = a + b$ assigns value of $a + b$ into c
+= Add AND	It adds right operand to the left operand and assign the result to left operand	$c += a$ is equivalent to $c = c + a$
-= Subtract AND	It subtracts right operand from the left operand and assign the result to left operand	$c -= a$ is equivalent to $c = c - a$
*= Multiply AND	It multiplies right operand with the left operand and assign the result to left operand	$c *= a$ is equivalent to $c = c * a$
/= Divide AND	It divides left operand with the right operand and assign the result to left operand	$c /= a$ is equivalent to $c = c / a$ $c /= a$ is equivalent to $c = c / a$
%= Modulus AND	It takes modulus using two operands and assign the result to left operand	$c \% = a$ is equivalent to $c = c \% a$
**= Exponent AND	Performs exponential (power) calculation on operators and assign value to the left operand	$c ** = a$ is equivalent to $c = c ** a$
//= Floor Division	It performs floor division on operators and assign value to the left operand	$c //= a$ is equivalent to $c = c // a$

Lógicos

Operator	Description	Example
and Logical AND	If both the operands are true then condition becomes true.	(a and b) is true.
or Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is true.
not Logical NOT	Used to reverse the logical state of its operand.	Not(a and b) is false.

Inclusión

Operator	Description	Example
in	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	x in y, here in results in a 1 if x is a member of sequence y.
not in	Evaluates to true if it does not finds a variable in the specified sequence and false otherwise.	x not in y, here not in results in a 1 if x is not a member of sequence y.

Por ejemplo, lo siguiente devuelve True:

```
lst=[3,4,1,8] #lista
```

```
s=4
```

```
s in lst #devuelve True
```


Identidad

Operator	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y, here is results in 1 if id(x) equals id(y).
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here is not results in 1 if id(x) is not equal to id(y).

Por ejemplo:

```
a=3
```

```
b=5
```

```
c="x"
```

```
a is b #True
```

```
a is c #False
```

Imprimir resultados y leer datos

Como hemos visto, python dispone de la función print() para mostrar por pantalla un mensaje o incluso los resultados de una operación.

Mediante la función input() podemos solicitar un dato y recuperarlo dentro de un programa. Por ejemplo, la siguiente instrucción muestra un mensaje y guarda en la variable el dato introducido por el usuario:

```
dato=input("Introduce nombre")
```

Instrucción if

Evalúa una condición o expresión. En caso de ser una expresión, se interpreta como True si el resultado es distinto de 0 o no nulo, mientras que se interpretará como falso si es 0 o nulo.

No se usan paréntesis para encerrar la condición/expresión y se deben poner dos puntos (:) a continuación de esta:

```
a=3
```

```
b=2
```

```
if a==b : print("iguales")
```

```
else: print("distintos")
```

Lo anterior imprime *distintos*

Si el bloque se compone de más de una instrucción, se separan por punto y coma (;):

```
if a: print("iguales");print("salida")
```

Lo anterior imprime *iguales salida*

Se pueden poner las instrucciones de cada bloque en líneas independientes, pero en este caso será necesario indentarlas:

```
if a==b:
    print("iguales")
    print("fin")
else:
    print("distintos")
```

Si alguna de las instrucciones **no está indentada se produce un error**.

Instrucción while

Ejecuta el bloque de instrucciones indentadas mientras se cumpla la condición. La condición puede escribirse entre paréntesis o no. El siguiente código de ejemplo muestra los números del 1 al 10:

```
conta=1;
while(conta<=10):
    print(conta)
    conta=conta+1
```

Instrucción for

Se emplea para iterar una secuencia de datos. En el siguiente ejemplo vemos como usar un for para mostrar el contenido de una lista de nombres:

```
dias=["lunes","martes","miércoles","jueves","viernes"]
for v in dias:
    print(v)
```

También puede utilizarse para recorrer la lista con un índice:

```
finde=["sabado","domingo"]
```

```
for i in range(len(finde)):
```

```
    print(finde[i])
```

Tanto while como for soportan el uso de las sentencias *break* y *continue*.

Si lo que queremos es simplemente recorrer un rango numérico, podemos utilizar la función `range` con los valores extremos. El siguiente ejemplo imprime los números del 1 al 10:

```
for i in range(1,10):
```

```
    print(i)
```

Cadenas de caracteres

En python no existe el tipo `caracter`, solo el tipo `cadena`. Un `caracter` sería una `cadena` de longitud 1. Las cadenas se representan entre comillas dobles:

```
cad="Esto es una cadena"
```

Se puede acceder a una subcadena utilizando los corchetes:

```
cad="Esto es una cadena"
```

```
print ("primer caracter: ",cad[0]) #E
```

```
print ("rango: ",cad[1:6]) #sto e
```

```
print ("primeros: ",cad[:3]) #Est
```

Se puede utilizar el operador `+` para concatenar cadenas y el `*` para repetirlas:

```
otra="java"
```

```
print(cad+otra*2) #Esto es una cadenajavajava
```

Métodos de string

A continuación, comentamos algunos de los métodos más interesantes que se pueden aplicar sobre una cadena de caracteres:

- `count()`. Indica cuantas veces aparece una cadena dentro de otra. Además de la cadena a buscar, se indica el rango de posiciones de búsqueda:

```
cad="Esto es una cadena de estas"
```

```
print(cad.count("es",0,24)) # 2
```

- `endsWith`. Indica si la cadena termina con la subcadena que se le pasa como parámetro. Se puede incluir el rango de posiciones de búsqueda:

```
muestra="na"
print(cad.endswith(muestra)) #false
print(cad.endswith(muestra,0,11)) #true
print(cad.endswith(muestra,0,18)) #true
```

- `find()`. Busca una subcadena dentro de otra, devolviendo su posición. Si no la encuentra devuelve -1. Se puede incluir el rango de posiciones de búsqueda:

```
print(cad.find("una")) #8
print(cad.find("una",10)) #-1, pues comienza a partir de la pos 10
```

- `isalpha()`. Devuelve true si todos los caracteres de la cadena son alfabéticos, false si alguno no lo es
- `isdigit()`. Devuelve true si todos los caracteres de la cadena son numéricos
- `isalnum()`. Devuelve true si todos los caracteres son alfanuméricos
- `len()`. Se utiliza como una función, devuelve la longitud de la cadena que se le pasa como parámetro:

```
cad="mi cadena"
print(len(cad)) #9
```

- `split()`. Convierte una cadena en una lista formada por una serie de items, utilizando como separador la cadena que se proporciona como parámetro (por defecto es el espacio):

```
dias="lunes,martes, miercoles, jueves"
print(dias.split(",")) #['lunes', 'martes', ' miercoles', ' jueves']
```

Conversión de cadena a entero

Una cadena formada por caracteres numéricos se puede convertir a número mediante la función `int()`. Esta función es muy útil cuando se quiere leer un número por teclado, ya que la función `input()` lo entrega como texto:

```
num=int(input("Introduce un número"))
```